B.Sc. in Computer Science and Engineering Thesis

# A Supervised Learning Approach to Determine Optimum Leader Election Algorithm in Topology-Aware Distributed Systems


Submitted by

Nabila Tabassum
202005050

Azmal Karim
202005056

Supervised by

Dr. Rezwana Reaz

**Department of Computer Science and Engineering**
**Bangladesh University of Engineering and Technology**

Dhaka, Bangladesh


June 2026

# CANDIDATES' DECLARATION

This is to certify that the work presented in this thesis, titled, "A Supervised Learning Approach to Determine Optimum Leader Election Algorithm in Topology-Aware Distributed Systems", is the outcome of the investigation and research carried out by us under the supervision of Dr. Rezwana Reaz.

It is also declared that neither this thesis nor any part thereof has been submitted anywhere else for the award of any degree, diploma or other qualifications.

---

Nabila Tabassum
202005050

---

Azmal Karim
202005056

# ACKNOWLEDGEMENT

We are deeply grateful to the Almighty for blessing us with the wisdom, health, and perseverance to undertake and complete this research.

We would like to express our heartfelt gratitude to our supervisor, Dr. Rezwana Reaz, for her unwavering support, valuable guidance, and continuous encouragement throughout this journey. Her mentorship has been a cornerstone in shaping this work, and we couldn't have done it without her.

We also want to express our sincere appreciation to Md. Jehadul Karim, Nur Uddin Saimon and Mobaswirul Islam for their support, guidance, and encouragement. Their insightful advice have been essential to the completion of this research.

Finally, we are deeply thankful to our classmates, friends, and family for their unending support and motivation. Their encouragement has been a constant source of strength, pushing us forward even during challenging times.

Dhaka
June 2026

Nabila Tabassum

Azmal Karim

# Contents

*CANDIDATES' DECLARATION* i

*ACKNOWLEDGEMENT* ii

**List of Figures** v

**List of Tables** vi

*ABSTRACT* vii

**1 Introduction** 1

1.1 Introduction . . . 1
1.2 Objectives . . . 2
1.3 Our Contributions . . . 2
1.4 Thesis Organization . . . 3

**2 Preliminaries** 4

2.1 Distributed Systems . . . 4
2.2 Leader Election . . . 5
2.3 Leader Election Algorithms . . . 5
2.3.1 Bully Algorithm . . . 5
2.3.2 Ring Algorithm . . . 6
2.3.3 Bidirectional Ring Algorithm . . . 6
2.3.4 Echo Algorithm . . . 6
2.3.5 Raft Algorithm . . . 7
2.4 Supervised Learning . . . 7
2.4.1 Classification and Regression . . . 7
2.4.2 Linear Regression . . . 8
2.4.3 Logistic Regression . . . 8
2.4.4 Deep Learning . . . 8

**3 Literature Review** 10

3.1 Related Work . . . 10
3.2 Classical Leader Election Algorithms . . . 10

3.3 Machine Learning Approaches to Leader Election . . . 12
3.4 Topology-Aware Leader Election . . . 12
3.5 Performance-Aware and Adaptive Approaches . . . 13

**4 Methodology** **14**
4.1 Scenario Input Generation . . . 14
4.2 Graph Construction . . . 15
4.2.1 Find Hamiltonian cycle for Ring and Bidirectional Ring . . . 15
4.3 Failure and Network Model . . . 16
4.3.1 Failure Schedule Generation . . . 16
4.3.2 Election Trigger Generation . . . 16
4.3.3 Network Model . . . 17
4.4 Leader Election Simulation . . . 17
4.5 Performance Metric Collection . . . 18
4.6 Machine Learning Model Training . . . 19
4.6.1 Relative Error of Incorrect Predictions . . . 20
4.7 Adaptive Leader Election Algorithm Selection . . . 21

**5 Experiments and Results** **23**
5.1 Experimental Metrics . . . 23
5.2 Results . . . 24
5.2.1 Relationship Between Optimal LEA and Graph Metrics . . . 24
5.2.2 Machine Learning Model Accuracy . . . 27
5.2.3 Feasibility of Dynamic LEA Selection . . . 28
5.3 Findings . . . 29

**6 Future Work** **30**

**7 Conclusion** **32**

**8 Discussion** **34**

**References** **37**

# List of Figures

4.1 Overall workflow of the proposed framework. . . . . 22

5.1 Relationship between number of nodes and best LEA (for time). . . . . 25
5.2 Relationship between number of nodes and best LEA (for message count). . . . 25
5.3 Relationship between average degree and best LEA. . . . . 26
5.4 Relationship between packet drop rate and best LEA. . . . . 26
5.5 Learning curve for logistic regression model. . . . . 27
5.6 Learning curve for deep learning model . . . . . 28

# List of Tables

5.1 Model Performance Evaluation . . . . . 27
5.2 Performance Comparison of Leader Election Strategies . . . . . 29

# ABSTRACT

Leader election is a fundamental problem in distributed systems, where a coordinator is selected to manage and coordinate network activities. Numerous leader election algorithms have been proposed, including Bully, Ring, Bidirectional Ring, Raft, and Echo-based approaches. However, their performance varies depending on network characteristics such as topology, bandwidth, latency, node failures, and message loss, making it difficult to identify a single algorithm that performs optimally in all environments. This work presents a comparative study and machine learning–based selection framework for leader election algorithms in dynamic distributed systems. A simulated environment was developed to evaluate five leader election algorithms under diverse network conditions. The generated dataset contains performance metrics such as election time and message overhead across thousands of network configurations. Using this dataset, several machine learning models were trained to predict the most suitable leader election algorithm based on observable network characteristics. Experimental results show that no single algorithm consistently outperforms others across all network environments. Instead, algorithm effectiveness depends strongly on current network conditions. The proposed machine learning–based framework enables adaptive algorithm selection and achieves improved overall performance compared to using a fixed leader election algorithm. These findings demonstrate the potential of combining machine learning with distributed systems techniques to enhance leader election efficiency in dynamic network environments.

# Chapter 1

# Introduction

## 1.1 Introduction

Leader election is a fundamental problem in distributed systems, where a single coordinator node is selected among a group of distributed processes to manage and coordinate network activities. This coordinator typically handles critical responsibilities such as task scheduling, resource allocation, and maintaining consistency across the network. As distributed systems have grown in scale and complexity, the importance of efficient and reliable leader election has only increased - failures in this process can cascade into broader system instability, making the choice of election algorithm a matter of significant practical concern.

Over the years, numerous leader election algorithms have been proposed, each grounded in different design philosophies and suited to different operating assumptions. Classical approaches such as the Bully algorithm rely on direct node-to-node communication and assume nodes are aware of each other's identities. Ring-based algorithms, including unidirectional and bidirectional variants, organize nodes into a logical topology and circulate election tokens around it. More modern approaches such as Raft incorporate leader election as part of a broader consensus mechanism, while Echo-based algorithms leverage spanning tree constructions to propagate election information across arbitrary network topologies.

Despite this diversity of approaches, a persistent challenge remains: no single algorithm performs optimally under all network conditions. Performance varies significantly depending on factors such as network topology, available bandwidth, communication latency, the rate of node failures, and the probability of message loss. An algorithm that performs efficiently in a stable, low-latency network may incur excessive message overhead or prolonged election times when failures are frequent or the network is heavily partitioned. This sensitivity to environmental conditions makes algorithm selection a non-trivial decision in practice, particularly in dynamic distributed systems where network characteristics can shift unpredictably over time.

This context-sensitivity motivates a natural question: rather than committing to a fixed election algorithm, can the right algorithm be chosen adaptively based on the observable state of the network? Answering this question requires not only a thorough understanding of how each algorithm behaves across diverse conditions, but also a principled framework for translating those observations into reliable selection decisions.

## 1.2 Objectives

The primary goal of this research is to explore the feasibility of adaptive leader election algorithm through performance analysis and machine learning–based algorithm selection. The specific objectives of this study are:

- To systematically evaluate how different leader election algorithms behave across diverse network conditions, and to understand the factors that drive performance differences among them.
- To investigate whether machine learning models can accurately predict the most suitable leader election algorithm for a given network state based on observable characteristics alone.
- To determine whether a prediction-driven, dynamic selection approach can yield measurable improvements in leader election performance compared to relying on any single fixed algorithm.

## 1.3 Our Contributions

- Present a comprehensive performance evaluation of multiple Leader Election Algorithms (LEAs) under various network conditions.
- Generate a simulation-based dataset to analyze the empirical behavior of these algorithms and compare their observed performance with theoretical expectations.
- Train machine learning models using these dataset to predict the most suitable LEA for a given network state.
- Demonstrate the feasibility of a dynamic and network-condition-aware approach to elect leader.

## 1.4 Thesis Organization

This thesis is organized as follows:

- **Chapter 2:** Preliminaries
- **Chapter 3:** Literature Review
- **Chapter 4:** Methodology
- **Chapter 5:** Experiments and Results
- **Chapter 6:** Future Work
- **Chapter 7:** Conclusion
- **Chapter 8:** Discussion

# Chapter 2

# Preliminaries

## 2.1 Distributed Systems

A distributed system is a collection of independent computers connected over a network, working together to present themselves as a single coherent system to whoever is using them. Unlike a traditional setup where everything runs on one machine, these systems spread their components across many physical or virtual nodes that might be sitting in different data centres or even different continents. The nodes have no shared memory and no common clock; the only way they can talk to each other is by sending messages over the network.

This architecture powers most of what we consider modern computing. Cloud platforms like AWS and Google Cloud, distributed databases like Cassandra and Spanner, blockchain networks, IoT deployments, and the large web services people use every day are all built on top of distributed systems. The scale at which these systems need to operate makes a centralised design simply impractical.

A few core properties shape how distributed systems are designed. *Scalability* is the ability to handle a growing workload by bringing in more nodes rather than upgrading a single machine. *Fault tolerance* means the system keeps doing its job correctly even when some of its nodes go down. *Availability* means requests are always answered. *Consistency* means every node agrees on the same data at the same time. These properties pull against each other in practice, and the CAP theorem captures this tension precisely: a distributed system can satisfy at most two of consistency, availability, and partition tolerance at the same time.

Getting the nodes to coordinate is one of the harder problems in this space. There is no central authority telling anyone what to do, so the nodes have to reach agreement among themselves on shared state, event ordering, and who is responsible for what. Mutual exclusion, distributed consensus, clock synchronisation, and leader election are all instances of this coordination problem. Failures make it harder still. Any node can crash at any time, messages can be delayed or

dropped, and the system needs to handle all of this without anyone stepping in to fix it manually.

## 2.2 Leader Election

In a distributed system, certain tasks require a single node to act as the coordinator. Replication, scheduling, and conflict resolution are all easier when one node is in charge. But you cannot simply fix that role to a particular machine, since that machine might fail. The system needs a way to agree on a coordinator dynamically and to replace that coordinator whenever it goes down.

A *Leader Election Algorithm* (LEA) is the mechanism that handles this. When an election is triggered, the nodes exchange messages according to some defined protocol and eventually converge on a single node as the new leader. The elected leader then takes over coordination duties until it too becomes unavailable, at which point another election runs.

Most algorithms pick the leader based on node identifiers or some notion of priority. The goal is always the same: arrive at a unique, agreed-upon leader quickly, with as little message overhead as possible, and do so even when some nodes are down or some messages are lost. Different algorithms make different trade-offs to achieve this, and no single algorithm is optimal for every network condition.

## 2.3 Leader Election Algorithms

### 2.3.1 Bully Algorithm

The Bully algorithm was proposed by García-Molina [1] in 1982. The rule it enforces is simple: whichever node has the highest identifier, and is still alive, becomes the leader.

When a node notices that the current leader has stopped responding, it kicks off an election by sending `ELECTION` messages to every node with a higher ID. Any such node replies with `OK`, meaning it is taking over, and the original sender backs off. The higher node then sends its own `ELECTION` messages to nodes above it, and this process repeats upward. A node that sends out `ELECTION` messages and receives no reply within a timeout declares itself the winner and broadcasts a `COORDINATOR` message to everyone. Multiple nodes can start elections at the same time; because higher-ID nodes always dominate, the process still converges on the right answer.

The algorithm is correct and straightforward to implement. The downside is message complexity: $O(n^2)$ in the worst case. There is also the "bully effect" — a node that just recovered can

immediately reclaim leadership and potentially trigger another round of unnecessary elections.

### 2.3.2 Ring Algorithm

In the Ring algorithm, introduced by Chang and Roberts [2], the nodes are arranged in a logical circle. An initiator starts the election by sending a token carrying its own ID around the ring. Each node that receives the token compares the ID in the token with its own, keeps whichever is larger, and passes it along. By the time the token has gone all the way around, it carries the highest ID in the network. That node wins, and a `COORDINATOR` token is sent around to let everyone know.

Message complexity under fault-free conditions is $O(n)$. The algorithm scales reasonably well and handles failed nodes by routing tokens to the next alive neighbour. The initiator is a single point of failure during the merge step; if it crashes before the result is finalised, the election has to start over.

### 2.3.3 Bidirectional Ring Algorithm

The Bidirectional Ring algorithm, due to Hirschberg and Sinclair [3], is a natural extension of the above. Instead of sending one token around the full ring, the initiator splits the ring in half and sends tokens in both directions simultaneously. Each token carries the election ID, the current maximum ID seen, a direction flag, and a hop counter set to roughly half the ring size. When a token's counter reaches zero it reports back to the initiator. Once both tokens have reported, the initiator picks the highest ID from the two results, elects that node, and broadcasts `COORDINATOR` tokens in both directions.

The bidirectional approach roughly halves the election latency compared to the single-direction version while keeping total message complexity at $O(n)$. The risks are the same as the unidirectional ring: the initiator is still a potential single point of failure, and the hop counters need to be set correctly or a segment might not cover all alive nodes.

### 2.3.4 Echo Algorithm

The Echo algorithm [4] takes a different approach and works on arbitrary connected graph topologies rather than requiring a ring. Every node starts an election simultaneously by sending its own ID to all its neighbours. When a node receives a wave carrying a higher ID than any it has seen before, it adopts that wave and forwards it to all its other neighbours. A wave carrying a lower ID is simply dropped. When a node has forwarded the winning wave to all its neighbours and received echo replies back from all of them, it sends a single echo reply toward whoever

originally sent it the winning wave. This process unwinds back to the source of the highest-ID wave, and once that node has collected echoes from all its neighbours, it declares itself the leader.

Message complexity is $O(n + e)$, where $e$ is the number of edges, since each edge is traversed at most twice. The algorithm's main weakness is that it has no built-in way to handle a node crashing mid-election. If any node fails before sending its echo, the originator waits indefinitely. The usual fix is a timeout that declares the election invalid and restarts it, but this gets expensive if failures happen often.

### 2.3.5 Raft Algorithm

Raft was designed by Ongaro and Ousterhout [5] to be a more approachable alternative to Paxos. The system at any moment has one leader, and all the others are followers. Followers listen for periodic heartbeats from the leader. If a follower goes too long without hearing one, it promotes itself to candidate, increments its term counter, and sends `RequestVote` messages to the rest of the cluster. A candidate that gets a majority of votes becomes the new leader for that term and immediately starts sending heartbeats to keep others from starting their own elections.

All writes go through the leader, which appends entries to its log and replicates them to followers before marking them as committed. If any node sees a message from a higher term than its own, it immediately steps down to follower. Persistent storage for the current term and last vote ensures that safety guarantees survive crashes and restarts.

Raft guarantees that committed entries are never lost and are applied in the same order on every non-faulty node. Its main practical costs are the storage I/O on every vote update and the fact that the leader is a replication bottleneck under heavy write traffic.

## 2.4 Supervised Learning

### 2.4.1 Classification and Regression

Supervised learning is a branch of machine learning where a model is trained on labelled examples. Each training example consists of an input and a known correct output; the model learns by adjusting its parameters to bring its predictions closer to those outputs, and the hope is that it then generalises well to inputs it has not seen before.

The two fundamental task types in supervised learning are *regression* and *classification*. In regression, the output is a continuous number. Predicting how long a leader election will take or how many messages it will generate are regression problems. In classification, the output is a

discrete category. Predicting *which* algorithm will perform best for a given network configuration is a classification problem. Our work uses both: regression models predict raw performance metrics, while classification models predict the optimal algorithm label directly.

### 2.4.2 Linear Regression

Linear regression is the simplest supervised model. It assumes the output can be written as a weighted sum of the input features:

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b \tag{2.1}$$

where $\mathbf{w}$ is a weight vector and $b$ is a scalar bias. Training means finding the $\mathbf{w}$ and $b$ that minimise the mean squared error between predictions and true labels, which has a clean closed-form solution. The model is fast to fit and easy to inspect, but it can only capture linear relationships. When the true relationship between inputs and output involves non-linear interactions, a linear model will consistently under-fit, no matter how much data it sees.

### 2.4.3 Logistic Regression

Logistic regression is a classification algorithm despite its name. It passes a linear combination of the input features through the logistic function to produce a probability:

$$P(y = 1 \mid \mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}} \tag{2.2}$$

For problems with more than two classes, the softmax function generalises this to a full probability distribution over all categories. Training minimises the cross-entropy loss using gradient descent. Logistic regression is computationally cheap, interpretable, and often works well when the decision boundaries happen to be roughly linear in the feature space. It is a natural baseline to compare against before moving to more complex models.

### 2.4.4 Deep Learning

A deep learning model, specifically a multilayer perceptron (MLP) in our case, stacks multiple layers of learned transformations. Each layer applies a linear mapping followed by a non-linear activation function:

$$\mathbf{h}^{(l)} = \sigma\big(\mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\big) \tag{2.3}$$

where $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are learnable parameters at layer $l$ and $\sigma$ is an activation function such as ReLU. Stacking several such layers lets the network build up increasingly abstract representations of the input, which is what allows it to model relationships that a linear model simply cannot express. Parameters are learned via backpropagation, which computes gradients of the loss with respect to every weight in the network and applies them through stochastic gradient descent.

The cost of this expressiveness is a larger parameter count, higher data requirements, and more decisions to make during training, such as network depth, layer width, learning rate, and regularisation. When the underlying relationship is genuinely complex and enough data is available, an MLP tends to outperform simpler alternatives by a meaningful margin. In our experiments, predicting which leader election algorithm is optimal for a given network turns out to be exactly the kind of problem where this extra capacity pays off.

# Chapter 3

# Literature Review

## 3.1 Related Work

Leader election is one of the older problems in distributed systems, and there is no shortage of work on it. The core challenge has always been the same: given a group of nodes that can only communicate by passing messages, how do you get them to agree on a single coordinator, especially when some of those nodes might crash or messages might get lost? Over the years, several algorithms have been proposed to tackle this, each making different trade-offs. More recently, a handful of studies have tried to bring machine learning into the picture. We discuss the most relevant of these works below.

## 3.2 Classical Leader Election Algorithms

**Bully Algorithm.** Garcia-Molina [1] introduced the Bully algorithm, which takes a straightforward approach: whichever node has the highest identifier wins. When a node notices that the current leader has gone silent, it sends election messages to every node with a higher identifier. If someone responds, the initiator backs off and waits. If nobody responds within a timeout, the node declares itself the leader and notifies everyone. The algorithm is simple to understand and works well in small networks. The trouble is that it does not scale gracefully. As the number of nodes grows, the volume of election messages grows with it, and the algorithm ends up being one of the heavier options in terms of message overhead.

**Ring Algorithm.** Chang and Roberts [2] proposed a cleaner approach for systems where nodes can be arranged in a logical ring. A node that wants to start an election sends a token carrying its own identifier around the ring. Each node that receives the token compares it with its own identifier, keeps whichever is larger, and passes it along. By the time the token completes a full

trip around the ring, it carries the highest identifier in the network, and that node is declared the leader. The algorithm is elegant and, somewhat counterintuitively, tends to become more efficient as the network grows larger, since the number of messages per election is always proportional to the ring size.

**Bidirectional Ring Algorithm.** Hirschberg and Sinclair [3] extended the standard ring by allowing nodes to hold different weights or priorities rather than relying purely on identifiers. This makes it possible to favour nodes with better hardware or more stable connectivity when electing a leader. The trade-off is that the election process becomes more computationally involved, since each node needs to evaluate weighted criteria rather than doing a simple comparison. In practice, unless the application genuinely needs weighted leadership, this variant tends to be slower than the original Ring or even the Bully algorithm.

**Echo Algorithm.** The Echo algorithm [4] belongs to a family of protocols known as wave algorithms, which work by flooding a message outward from an initiator and then collecting acknowledgments as the wave bounces back. The initiator sends probe messages to its neighbours, which forward them further until the entire network has been reached. The echoes then carry information back to the initiator, such as the highest identifier seen across all nodes, which is then elected as the leader. The algorithm works on arbitrary topologies, which is a meaningful advantage over ring-based methods. The downside is latency: the multi-phase structure means elections take longer to complete compared to simpler approaches.

**Paxos.** Lamport [6] introduced Paxos as a general solution to the consensus problem in asynchronous systems where nodes can crash at any time. Leader election in Paxos is a byproduct of the broader consensus mechanism: the protocol ensures that a majority of non-faulty nodes eventually agree on a single coordinator, even in the presence of failures and message delays. Systems like ZooKeeper and Kubernetes rely on Paxos-based coordination under the hood. The protocol is provably correct, but its complexity is well-known; Lamport himself acknowledged that it is difficult to understand and even harder to implement correctly.

**Raft.** Raft was designed by Ongaro and Ousterhout [5] specifically because Paxos was so hard to work with in practice. The key idea is that one node acts as a strong leader at all times, handling all log writes and sending periodic heartbeats to followers. If a follower stops hearing from the leader for too long, it starts an election by requesting votes from its peers. To avoid multiple nodes starting elections simultaneously, each node waits a random amount of time before timing out. This randomisation is simple but effective, and Raft has become the go-to choice for many modern distributed systems precisely because its behaviour is much easier to reason about than Paxos.

## 3.3 Machine Learning Approaches to Leader Election

Dhariwal [7] was among the first to apply machine learning to the problem of leader election algorithm selection. The study trained linear regression models to predict which of three algorithms — Bully, Ring, or Modified Ring — would be fastest for a given system size. Using C++ implementations with MPI on a 16-core machine, experiments were run across networks of 5 to 300 nodes, and the models achieved a prediction accuracy of 94% for execution time.

The work is a useful starting point, but it has some important gaps. The experiments were run on a single multi-core machine using shared memory, so the results do not reflect what happens when messages actually travel across a real network with finite bandwidth and non-zero loss rates. The model also uses only the number of nodes as its input feature, which means anything to do with topology shape, packet drops, or node failures is simply not captured. For a controlled, homogeneous environment that is fine, but real distributed systems are rarely so well-behaved. Our work picks up where this study leaves off by building a simulation that explicitly models bandwidth constraints, message drop rates, and node failures, and by extending the comparison to five algorithms instead of three.

## 3.4 Topology-Aware Leader Election

Favier et al. [8] took a different angle on the problem, focusing specifically on mobile ad hoc networks where the topology is constantly changing. Rather than electing whichever node has the highest identifier, their algorithm elects the node with the highest closeness centrality, meaning the node that is most reachable from the rest of the network. Every node maintains a local model of the network graph, which it keeps up to date through periodic broadcasts using logical clocks. The result is an algorithm that makes topologically informed decisions rather than arbitrary ones.

The approach is interesting, but the overhead it requires is hard to ignore. Maintaining and continuously refreshing a global network view at every node is expensive. The paper reports that message sizes in their protocol can exceed 1,200 bytes, compared to just 196 bytes for a simpler flooding approach, because each message carries graph map data. The system also has a rough startup period where the leader error rate can be as high as 69% before the local network views have converged. These are not trivial costs, and they raise questions about whether the topological awareness is worth the price in resource-constrained or bandwidth-limited environments.

## 3.5 Performance-Aware and Adaptive Approaches

Sakic et al. [9] proposed SEER, a system that monitors the performance of the current leader in real time and triggers a re-election if a better candidate is available. Rather than waiting for a node to crash before holding an election, SEER proactively replaces underperforming leaders with faster ones. This is a meaningful departure from traditional protocols, which treat leadership changes as responses to failure rather than opportunities for optimisation.

SEER sits at an interesting intersection of the ideas in Dhariwal [7] and Favier et al. [8]. Like Dhariwal, it uses predictive logic to avoid suboptimal choices. Like Favier, it takes network conditions into account rather than relying on static identifiers. The limitation is that SEER still operates within a single fixed protocol; it picks a better node to be the leader, but it does not consider whether a different algorithm might be more appropriate for the current network state altogether.

That distinction is where our work diverges from the existing literature. None of the studies above treat the algorithm itself as the variable to be optimised. We argue that for a dynamically changing network, the question of which algorithm to run is just as important as the question of which node should win that election. By training supervised ML models on data collected from thousands of simulated network configurations, we let the model learn when Raft is the right choice, when Ring is better, and when the topology is so unstable that even a simple Bully run is preferable. The result is a framework for algorithm-level adaptation, not just coordinator-level adaptation.

# Chapter 4

# Methodology

## 4.1 Scenario Input Generation

We started by generating a diverse set of network graphs representing different distributed system topologies. The graphs was generated by varying network topology and communication parameters.

- The number of nodes was selected from 3 to 51 with increments of 3 to cover small to medium-scale distributed systems while keeping simulation time manageable.
- For each network size, up to five edge counts were selected uniformly from the range $[n-1, \frac{n(n-1)}{2}]$, where $n-1$ represents the minimum number of edges required for a connected graph and $\frac{n(n-1)}{2}$ represents the maximum number of edges in a complete graph. This allowed the experiments to capture both sparse and dense network topologies.
- Bandwidth values of 0.1, 1, 10, 100, 1000 Mbps were selected to represent low-speed, moderate-speed, and high-speed communication links commonly found in real-world networks.
- The Latency parameter was evaluated using values of 5, 25, 100 second, representing low-latency local networks, medium-latency campus/regional networks, and higher-latency wide-area networks.
- The values of Failure rate for different nodes were chosen from 0.001, 0.005, 0.01, 0.025 to model rare to frequent node failures while maintaining realistic system availability.
- Loss rates of 0.01, 0.02, 0.05, 0.10 and 0.15 were considered to represent reliable, moderately lossy, and highly lossy network environments, allowing the evaluation of how packet loss and retransmissions affect leader election performance.

## 4.2 Graph Construction

Each experiment begins with a network configuration consisting of *num_nodes*, *num_edges*, *bandwidth*, *latency*, *failure_rate*, and *loss_rate*, which are stored as the scenario input. For each scenario, a connected undirected graph is generated. The graph generator first constructs a random spanning tree to guarantee connectivity and then adds additional random edges until the required number of edges is reached.

$$G = \texttt{random_connected_graph}(num_nodes,\ num_edges) \tag{4.1}$$

After graph generation, several structural properties of the graph are computed and recorded as input features:

- Graph density
- Average Degree
- *hamiltonian_cycle_found*

where V is the set of vertices, E is the set of edges. Graph Density is calculated using this formula

$$\text{Graph Density} = \frac{2|E|}{|V|(|V|-1)} \tag{4.2}$$

and Average Degree is calculated by

$$\text{Average Degree} = \frac{2|E|}{|V|} \tag{4.3}$$

### 4.2.1 Find Hamiltonian cycle for Ring and Bidirectional Ring

This is important because Ring and Bidirectional Ring algorithms can only run if the actual graph contains a Hamiltonian cycle. If no Hamiltonian cycle is found, Ring and Bidirectional Ring results are set to 0. The Hamiltonian cycle finder uses bounded backtracking and returns a cycle only if consecutive nodes and the last-first node pair are connected by real graph edges.

The Hamiltonian cycle indicator is defined as:

$$\textit{hamiltonian_cycle_found} = \begin{cases} 1, & \text{if } G \text{ contains a Hamiltonian cycle} \\ 0, & \text{otherwise} \end{cases} \tag{4.4}$$

## 4.3 Failure and Network Model

To ensure a fair comparison among leader election algorithms, a common failure schedule and network model were generated for each scenario. All algorithms were evaluated using the same node failures, recoveries and election trigger events.

### 4.3.1 Failure Schedule Generation

For each network scenario, a shared failure schedule was generated using the configured number of nodes, simulation duration, and failure rate:

$$\texttt{schedule} = \texttt{generate_failure_schedule}(\texttt{num_nodes}, \texttt{sim_time}, \texttt{failure_rate}) \tag{4.5}$$

The failure process was modeled using alternating crash and recovery events. The mean crash time was defined as

$$T_{\text{crash}} = \frac{1}{\texttt{failure_rate}} \tag{4.6}$$

and the mean recovery time was calculated as

$$T_{\text{recovery}} = 0.20 \times T_{\text{crash}}. \tag{4.7}$$

At the beginning of the simulation, all nodes were assumed to be alive. Each node then alternated between alive and crashed states according to exponentially distributed crash and recovery intervals. The same failure schedule was reused across all leader election algorithms to ensure that performance differences were attributable only to the algorithms themselves.

### 4.3.2 Election Trigger Generation

Election trigger events were derived directly from the shared failure schedule. Two rules were applied:

1. An initial leader election was triggered at time $t = 0$.
2. A new election was triggered whenever the current highest-ID alive leader crashed.

Consequently, the system initiated a new leader election only when the current leader became unavailable.

### 4.3.3 Network Model

A network model was created for each scenario using the configured latency, bandwidth, and message loss rate parameters. Communication delay consisted of both propagation and serialization delays. Higher latency values increased message propagation delay, whereas lower bandwidth values increased serialization delay. The election timeout was kept a fixed value for all the network.

## 4.4 Leader Election Simulation

A custom discrete-event simulation framework was developed in Python to evaluate the performance of leader election algorithms under varying network conditions. For each generated network scenario, the simulator combines the graph topology, network model, failure schedule, and election trigger events to create a complete distributed-system environment.

Each simulation run spans 5000 seconds of simulated time. During this period, nodes communicate through the network model and experience crash and recovery events according to the shared failure schedule. Whenever the current leader becomes unavailable, a new election is triggered based on the predefined election schedule.

To ensure a fair comparison, each leader election algorithm is executed independently using the same network topology, failure schedule, election triggers, bandwidth, latency, and message loss parameters. Thus, any observed performance differences arise solely from the behavior of the algorithms themselves rather than from variations in the simulation environment.

The following leader election algorithms were evaluated:

- Bully Algorithm
- Ring Algorithm
- Bidirectional Ring Algorithm
- Raft Leader Election
- Echo-Based Leader Election

The Ring and Bidirectional Ring algorithms require the existence of a Hamiltonian cycle in the network topology. Therefore, these algorithms were executed only for scenarios in which

a Hamiltonian cycle was detected. For scenarios without a Hamiltonian cycle, Ring-based algorithms were marked as unavailable.

For each simulation run, the framework records election-related events and performance metrics, including election completion time, message count, number of completed elections, and failed election attempts. These measurements form the basis of the dataset used for subsequent analysis and machine learning model training.

## 4.5 Performance Metric Collection

For each simulation run, the framework records several performance metrics that characterize the behavior of the leader election algorithm. These metrics include:

- Average election time
- Average message count
- Number of completed elections
- Number of failed election attempts

Only successful elections are considered when computing average election time and average message count. Let $N_s$ denote the number of successful elections, $T_i$ denote the completion time of the $i$-th successful election, and $M_i$ denote the corresponding message count.

The average election time is calculated as

$$\textit{avg_time} = \frac{\sum_{i=1}^{N_s} T_i}{N_s}. \tag{4.8}$$

Similarly, the average message count is calculated as

$$\textit{avg_msg_count} = \frac{\sum_{i=1}^{N_s} M_i}{N_s}. \tag{4.9}$$

In addition to these averages, the total number of completed elections and failed election attempts are also recorded. A completed election corresponds to an election that successfully elects a leader, whereas a failed attempt corresponds to an election that does not complete due to node failures, message loss, timeout expiration, or algorithm-specific constraints.

## 4.6 Machine Learning Model Training

The dataset generated by the simulation framework was used to train machine learning models for predicting the most suitable leader election algorithm (LEA) for a given network condition. Each row of the dataset corresponds to a single simulation scenario and contains the network characteristics together with the performance measurements obtained from all evaluated leader election algorithms.

The input feature vector consists of the following ten network attributes:

- Number of nodes
- Number of edges
- Graph density
- Average degree
- Number of connected components
- Bandwidth
- Latency
- Failure rate
- Loss rate
- Hamiltonian cycle indicator

For each leader election algorithm, the following performance metrics were recorded:

- Average election time
- Average message count
- Number of completed elections
- Number of failed election attempts

These performance metrics are then used to derive a unified performance score.

The performance score for an algorithm is defined as

$$score = 1.0 \times normalized_time + 1.0 \times normalized_msg_count + 2.0 \times normalized_failure. \quad (4.10)$$

The failure component is assigned a larger weight in order to place greater emphasis on election reliability. For each simulation scenario, the leader election algorithm with the lowest score is considered the optimal algorithm and is used as the target label for machine learning.

Before training, the dataset was divided into training and testing sets using an 80%-20% split. Input features were standardized using the statistics computed from the training data.

Three machine learning approaches were investigated:

- Linear Regression
- Logistic Regression
- Deep Learning

Linear Regression models were trained to predict the performance score of each leader election algorithm. The algorithm associated with the lowest predicted score was then selected as the recommended choice.

Logistic Regression and Deep Learning models were formulated as multi-class classification problems, where the target class represents the optimal leader election algorithm for a given network scenario.

The trained models were evaluated on previously unseen test data using classification accuracy, precision, recall, and F1-score. These metrics measure the ability of the models to correctly identify the most appropriate leader election algorithm under varying network conditions.

### 4.6.1 Relative Error of Incorrect Predictions

Traditional classification metrics such as accuracy, precision, recall and F1-score indicate how frequently a model predicts the correct leader election algorithm. However, these metrics do not capture the severity of incorrect predictions. In practice, selecting a near-optimal algorithm is considerably less harmful than selecting a poor-performing algorithm, even though both predictions are counted as incorrect.

To quantify the quality of incorrect predictions, an additional metric called *Relative Error* was used. For each incorrectly classified scenario, the performance score of the predicted algorithm was compared with the score of the optimal algorithm.

Let $S_{best}$ denote the score of the optimal algorithm, $S_{pred}$ denote the score of the algorithm predicted by the model, and $S_{worst}$ denote the score of the worst-performing algorithm for the same network scenario. The relative error is computed as

$$RE = \frac{S_{pred} - S_{best}}{S_{worst} - S_{best}}. \tag{4.11}$$

A value of $RE = 0$ indicates that the predicted algorithm performs identically to the optimal algorithm, whereas a value of $RE = 1$ indicates performance equivalent to the worst algorithm available for that scenario.

The average relative error over all incorrectly classified samples is then calculated as

$$AvgRE = \frac{1}{N_w} \sum_{i=1}^{N_w} RE_i, \tag{4.12}$$

where $N_w$ denotes the number of incorrect predictions.

This metric provides additional insight into model performance by measuring the practical cost of misclassification. A model with lower relative error is preferable because, even when it predicts the wrong leader election algorithm, the selected algorithm remains close to the optimal choice.

## 4.7 Adaptive Leader Election Algorithm Selection

Once trained, the machine learning model can be deployed to recommend the most suitable leader election algorithm without requiring the execution of all candidate algorithms. During deployment, a user only needs to provide the current network characteristics, namely the ten input features used during training. Based on these features, the model predicts the leader election algorithm that is expected to achieve the best overall performance under the given network conditions.

This approach eliminates the need to repeatedly execute and compare all leader election algorithms, thereby significantly reducing the computational overhead associated with algorithm selection.

For dynamic distributed systems, the network state is periodically monitored at fixed time intervals. At the end of each observation window, the current network characteristics are re-evaluated. If a change in topology or network conditions is detected, the updated feature values are supplied to the trained model, which then predicts the most appropriate leader election algorithm for the new network state.

Conversely, if no significant change is observed in the network characteristics, the currently selected leader election algorithm continues to operate and no re-evaluation is performed. This strategy avoids unnecessary algorithm switching while still allowing the system to adapt when

network conditions change.

As a result, the framework provides an adaptive mechanism for leader election algorithm selection that balances performance improvement and operational overhead in dynamic distributed environments.

The overall workflow of the proposed framework, from network scenario generation to adaptive leader election algorithm selection, is illustrated in Figure 4.1.

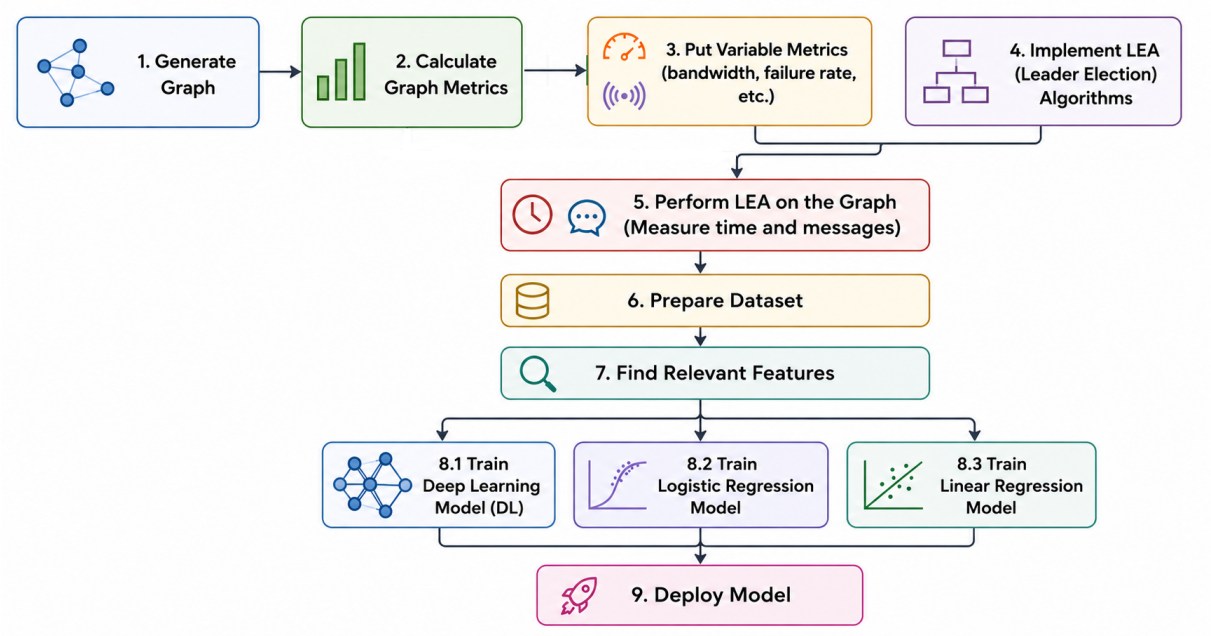



Figure 4.1: Overall workflow of the proposed framework.

# Chapter 5

# Experiments and Results

## 5.1 Experimental Metrics

To evaluate the effectiveness of different Leader Election Algorithms (LEAs), we used three performance metrics.

1. **Time**: The average amount of time required to complete a leader election process. This is the most intuitive metric and directly reflects election latency.
2. **Message Count**: The average number of messages exchanged during the election process. This metric is important because excessive message propagation may cause network congestion, increase communication overhead, and consume additional power.
3. **Score**: We propose a hybrid evaluation metric that incorporates not only execution time and message count, but also election failures. In practical environments, packet loss or topology mismatches may cause an election algorithm to fail. Therefore, failure must be considered when evaluating overall performance.

The score is calculated as:

$$\text{Score} = 1.0 \times \text{normalized_time} + 1.0 \times \text{normalized_msg_count} + 2.0 \times \text{normalized_failure}$$

where

$$\text{normalized}_X = \frac{X - \min(X)}{\max(X) - \min(X)}$$

All three values are normalized to the range $[0, 1]$, ensuring that differences in raw scale do not bias the comparison. A lower score indicates better overall performance.

The failure component is assigned a weight of two, which is double the weight of the time and message metrics. The relative weights in the equation are followed by our intuition, to provide more importance to the failure than other two parameters. However, it is slightly backed by our experiment, as we tried three different weights for normalized failures (1.0, 2.0 and 3.0), and 2.0 provided the best outcome of the three. This weighting reflects our intuition that election failures have a significantly greater impact on system reliability than moderate increases in execution time or communication overhead. Consequently, the machine learning models are trained to minimize the selected optimization metric.

## 5.2 Results

The experimental findings can be categorized into three major areas:

1. Relationship between the optimal LEA and graph/network characteristics.
2. Machine learning model accuracy.
3. Feasibility of dynamic LEA selection.

### 5.2.1 Relationship Between Optimal LEA and Graph Metrics

Several expected behaviors can be observed from the generated dataset. Since the Bully algorithm relies heavily on broadcasting election messages, its performance deteriorates as the number of nodes and communication links increases. Conversely, Raft was designed with reliability as a primary objective and therefore performs particularly well in environments with high packet loss rates. The Bully algorithm also holds an advantage over ring-based approaches due to its reliance on broadcast communication. In practice, broadcasting election messages concurrently across the network is often more resilient than passing a singular token or message sequentially around a logical ring, where a single node failure or packet drop can easily stall the entire election process.

The dataset confirms these theoretical expectations and also reveals several additional relationships between graph properties, network constraints, and the optimal leader election strategy. Figure 5.1 to 5.4 illustrates some of the most significant observed patterns.

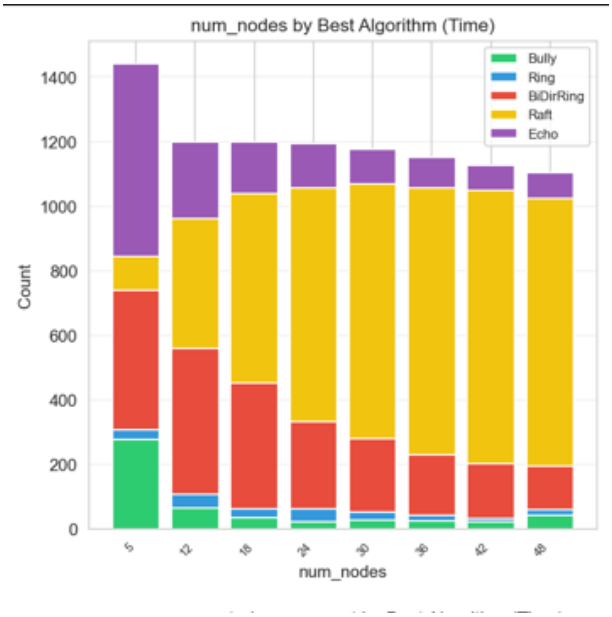



Figure 5.1: Relationship between number of nodes and best LEA (for time).

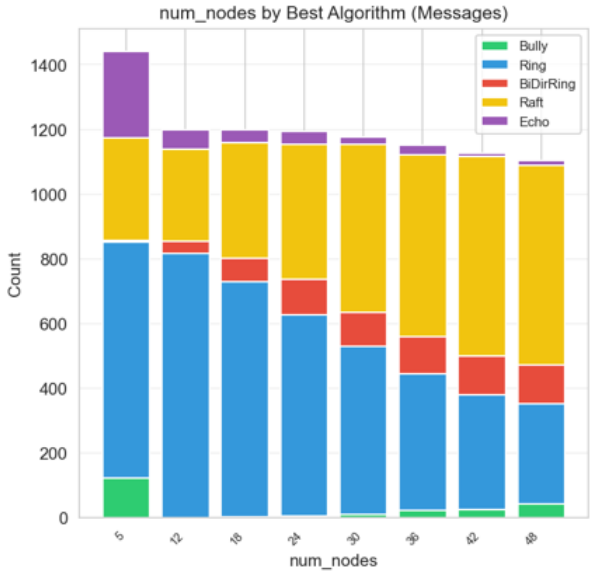



Figure 5.2: Relationship between number of nodes and best LEA (for message count).

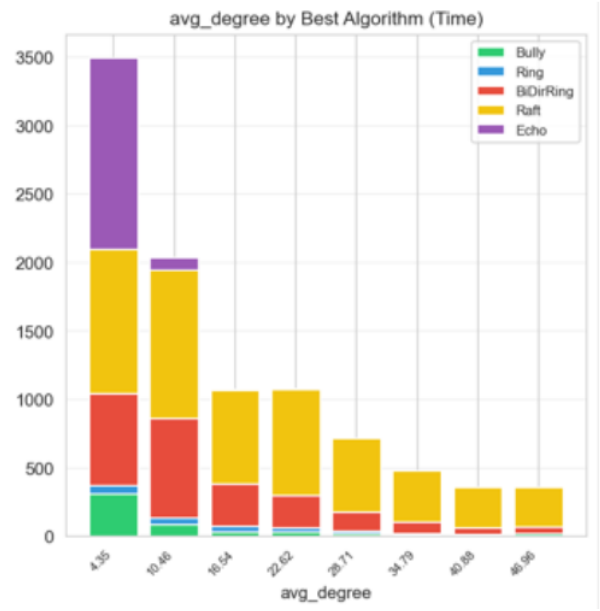



Figure 5.3: Relationship between average degree and best LEA.

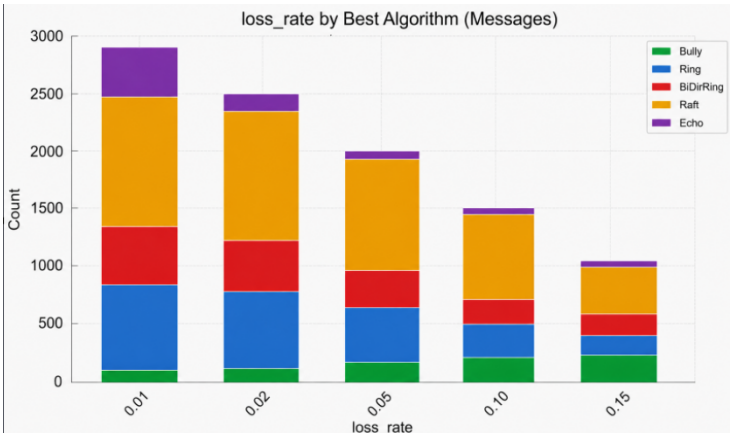



Figure 5.4: Relationship between packet drop rate and best LEA.

### 5.2.2 Machine Learning Model Accuracy

Three different machine learning approaches were evaluated:

- Linear Regression
- Logistic Regression
- Deep Learning

For each approach, three separate models were trained. One model attempted to optimize election time, another optimized message count, and the third optimized the proposed score metric.

The performance of these models is presented in Table 5.1.

Table 5.1: Model Performance Evaluation

| Model / Feature | Accuracy | F1-score | Relative Error |
|---|---|---|---|
| Linear Regression (Time) | 0.57 | 0.51 | 0.42 |
| Linear Regression (Messages) | 0.38 | 0.34 | 0.44 |
| Linear Regression (Score) | 0.74 | 0.71 | 0.25 |
| Logistic Regression (Time) | 0.77 | 0.74 | 0.21 |
| Logistic Regression (Messages) | 0.75 | 0.78 | 0.23 |
| Logistic Regression (Score) | 0.80 | 0.78 | 0.21 |
| Deep Learning (Time) | 0.81 | 0.79 | 0.21 |
| Deep Learning (Messages) | 0.79 | 0.77 | 0.22 |
| Deep Learning (Score) | 0.84 | 0.81 | 0.17 |

The learning curve of logistic regression model (score) and deep learning model (score) are given in Figure 5.5 and 5.6.

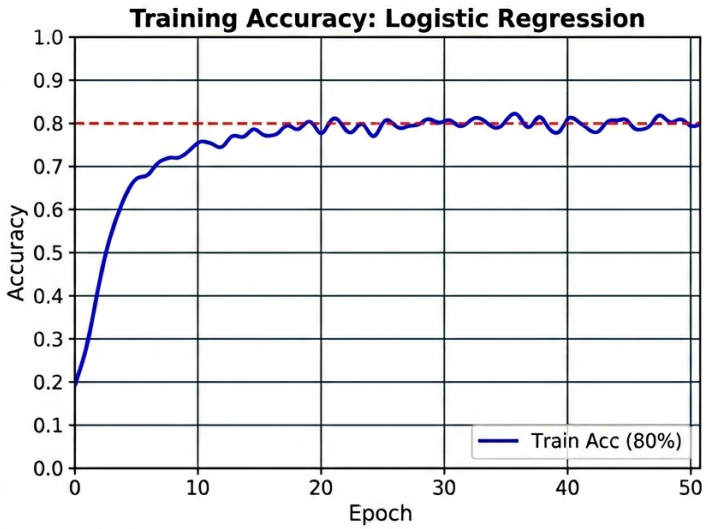



Figure 5.5: Learning curve for logistic regression model.

The results lead to several important observations:

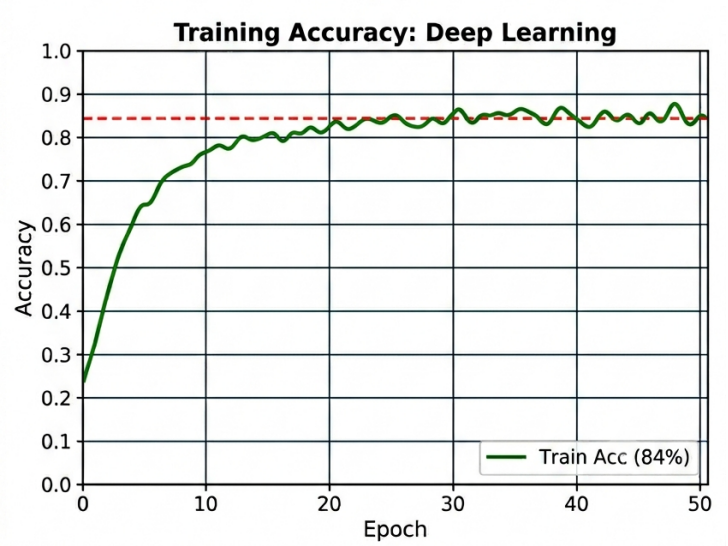



Figure 5.6: Learning curve for deep learning model

1. Linear regression models struggle to capture the complex and non-linear relationships that determine the optimal leader election algorithm.

2. Deep learning consistently achieves the best performance across all evaluation metrics, obtaining the highest accuracy and F1-score while maintaining the lowest relative error.

3. Logistic regression performs surprisingly well and remains competitive with deep learning. Considering its significantly lower computational requirements, it may represent a practical alternative in resource-constrained environments.

### 5.2.3 Feasibility of Dynamic LEA Selection

After training the machine learning models, they were deployed in a dynamic environment where the system automatically selected the most suitable leader election algorithm based on observed network characteristics.

Table 5.2 compares the performance of the dynamic approaches with fixed leader election strategies.

The results demonstrate that dynamic selection is a viable approach for leader election in distributed systems. While no single fixed algorithm performs optimally under all network conditions, the machine learning models are able to adaptively select appropriate strategies based on the characteristics of the underlying graph and network environment.

Among the evaluated approaches, the Deep Learning-based selector achieves the best overall score of 5.88, outperforming all static algorithms. Logistic Regression also performs remarkably well, achieving a score of 6.01 while requiring significantly less computational complexity.

Table 5.2: Performance Comparison of Leader Election Strategies

| Strategy | Avg Time | Avg Messages | Avg Score |
|---|---|---|---|
| Bully | 2.43 | 250.44 | 47.66 |
| Ring | 2.06 | 30.91 | 90.88 |
| Bidirectional Ring | 1.27 | 35.21 | 84.35 |
| Echo | 2.19 | 141.56 | 54.79 |
| Raft | 1.23 | 48.59 | 7.05 |
| Dynamic Linear Regression | 1.83 | 116.67 | 20.41 |
| Dynamic Logistic Regression | 1.27 | 42.96 | 6.01 |
| Dynamic Deep Learning*** | 1.18 | 53.87 | 5.88 |

These findings suggest that intelligent algorithm selection can substantially improve both efficiency and reliability in heterogeneous distributed environments.

## 5.3 Findings

Using Logistic Regression or Deep Learning methods, we obtain 80-84% accuracy for all three evaluation metrics. It proves that our model successfully predicts the best LEA for a network configuration.

Table II shows that the machine learning-based approach generally achieves better average performance than any single fixed leader election algorithm. The amount of improvement varies across different network conditions; in some cases the gain is marginal, while in others it is significant. After accounting for the computational overhead of the machine learning models, the adaptive approach still provides comparable or superior results in most scenarios. Based on these findings, we recommend dynamic algorithm selection for highly dynamic distributed systems where network characteristics frequently change, as the optimal leader election algorithm may vary over time. In contrast, for relatively stable environments where network conditions change infrequently, it may be more practical to use the machine learning model once initially to determine the most suitable leader election algorithm and then continue operating with that algorithm, thereby avoiding the overhead of repeated predictions, algorithm switching and extra power consumption.

# Chapter 6

# Future Work

While this work demonstrates the viability of a machine learning-based framework for adaptive leader election, several directions remain open for further investigation.

1. **Overhead Optimization**

   The current framework introduces additional computational and communication overhead through the prediction pipeline itself. Although the overhead is acceptable in the evaluated scenarios, it may become significant in resource-constrained distributed environments. Future work may investigate lightweight machine learning architectures, feature selection techniques, and model compression methods to reduce the computational cost of prediction while maintaining accuracy.

2. **Hybrid Selection Mode**

   The proposed framework selects a single leader election algorithm based on the predicted optimum strategy. However, network conditions may change during the election process itself. A potential extension is the development of a hybrid selection mode that combines the strengths of multiple leader election algorithms. Such a system could dynamically switch strategies during execution or utilize different algorithms for different phases of the election process. Designing such a mechanism while maintaining correctness and low coordination overhead presents an interesting research challenge.

3. **Expanded Topologies**

   The current evaluation relies on network configurations characterized by common graph-theoretic metrics. Although these experiments cover a broad range of scenarios, real-world distributed systems frequently exhibit more complex structures. Future studies could evaluate the framework on hierarchical networks, geographically distributed clusters, cloud-edge environments, and dynamically evolving topologies. Such investigations

would improve the generalizability of the framework and provide deeper insight into its behavior under realistic deployment conditions.

4. **Graph Neural Network-Based Solution**

   Graph Neural Networks (GNNs) have emerged as a powerful class of models for learning directly from graph-structured data [10–12]. Unlike traditional machine learning approaches that rely on handcrafted graph metrics, GNNs can automatically learn representations from the underlying network topology. Implementing a GNN-based prediction model therefore represents a particularly promising direction for future work. Such a model may capture complex structural relationships that are difficult to represent using scalar graph features alone, potentially improving the accuracy of leader election algorithm selection.

In summary, future research should focus on reducing prediction overhead, exploring hybrid leader election strategies, evaluating the framework on more diverse network topologies, and investigating graph neural network-based solutions. These directions have the potential to further improve the adaptability, efficiency, and robustness of machine learning-driven leader election in distributed systems.

# Chapter 7

# Conclusion

Leader election remains a fundamental challenge in distributed systems. The diversity of network topologies, communication constraints, and failure conditions makes it difficult for any single leader election algorithm to consistently achieve optimal performance across all deployment scenarios. Consequently, selecting a leader election algorithm as a static design decision often leads to suboptimal outcomes when network conditions evolve over time.

In this research, we proposed a supervised machine learning framework for topology-aware and adaptive leader election algorithm selection. To support this investigation, a discrete-event simulation environment was developed in Python that models five widely used leader election algorithms: Bully, Ring, Bidirectional Ring, Echo, and Raft. The simulator incorporates realistic network conditions, including varying bandwidth limitations, packet loss rates, and node failure probabilities. Through more than nine thousand experimental configurations, a comprehensive dataset was generated containing measurements of election completion time, message overhead, and election failures.

To evaluate algorithm performance holistically, we introduced a composite *Score* metric that combines normalized election time, message count, and failure rate. Since failed elections have a disproportionately negative impact on system stability, the failure component was assigned twice the weight of the other metrics. This scoring mechanism enabled meaningful comparison among algorithms operating under significantly different network conditions.

Using the generated dataset, three families of supervised learning models were trained: Linear Regression, Logistic Regression, and a Deep Learning model based on a multilayer perceptron architecture. Mutual information-based feature selection was employed to reduce the input space to the twelve most informative features. Experimental results showed that the Deep Learning model achieved the best overall performance, obtaining an accuracy of approximately 0.84, an F1-score of approximately 0.81, and a relative error of approximately 0.17. These findings indicate that the relationship between network characteristics and optimal algorithm

selection is highly non-linear and can be effectively learned using sufficiently expressive machine learning models.

To assess the practical value of adaptive selection, the trained models were deployed in a dynamic environment where nodes and communication links could appear or disappear throughout execution. The machine learning-based approaches were then compared against each static leader election algorithm. The Deep Learning-based selector achieved the best overall performance, producing an average Score of 5.88, surpassing the strongest static baseline, Raft, which achieved a Score of 7.05. These results validate the central hypothesis of this work: adaptive, context-aware algorithm selection can consistently outperform any fixed leader election strategy.

The findings also suggest several practical deployment recommendations. In relatively stable environments, the machine learning model may be executed once during system initialization to select the most suitable algorithm, thereby avoiding unnecessary inference overhead during operation. Conversely, in highly dynamic environments, periodic re-evaluation at each election event is beneficial, as the performance improvements outweigh the additional computational cost. Across all evaluated scenarios, the overhead introduced by prediction remained small compared to the gains in reliability and election efficiency.

Overall, this work demonstrates that supervised machine learning provides a viable and effective mechanism for intelligent leader election algorithm selection in distributed systems. By combining realistic network simulation, topology-aware feature engineering, and adaptive prediction models, the proposed framework offers a practical alternative to traditional static approaches. The results show that machine learning can successfully capture complex relationships between network conditions and algorithm performance, enabling more reliable and efficient leader election decisions. It is hoped that the methodologies, datasets, and findings presented in this work will serve as a foundation for future research in adaptive distributed systems and intelligent network management.

# Chapter 8

# Discussion

The results of this study support the central premise that no single leader election algorithm is universally optimal, and that a supervised machine learning model can learn when to use which one well enough to outperform any fixed choice. The dynamic deep learning strategy achieved a composite Score of 5.88 against Raft's 7.05, which was the best among fixed algorithms. That gap might look modest in absolute terms, but it is meaningful when you consider that Raft is already a well-engineered, reliability-focused protocol. Beating it consistently across a dynamic topology simulation is not a trivial result.

What the pattern analysis revealed is that the relationship between network conditions and optimal algorithm is genuinely non-trivial. Bully degrades predictably as node count grows, which matches its $O(n^2)$ message complexity. Raft holds up well under high packet loss, which is consistent with its design philosophy of prioritising reliability. But the crossover points, and the conditions under which Ring-based algorithms become competitive, are not easy to capture with a simple rule. That is precisely where the machine learning models add value. Linear regression, unsurprisingly, failed to capture these patterns. A linear model has no way to represent the interaction between, say, loss rate and graph density when those two factors jointly determine which algorithm wins. Logistic regression did considerably better, which suggests the classification boundaries are not wildly non-linear, just non-linear enough that a linear model cannot handle them. The deep learning model pushed the accuracy a bit further and reduced relative error to 0.17, which is where it earns its higher computational cost.

The composite Score metric deserves some discussion on its own, since it is a design choice that significantly shapes the conclusions. Weighting failures at $2\times$ relative to time and message count reflects a judgment that a failed election is qualitatively worse than a slow one. In most real systems, that judgment is defensible: a failed election means the cluster has no coordinator, which can halt progress entirely, whereas a slow election just delays it. But the specific weight of two is not derived from any theoretical argument; it is a reasonable engineering choice. Different weight assignments would shift the rankings, and future work could treat the weighting

itself as a tunable parameter fitted to specific deployment contexts.

One thing the results also show is that dynamic selection is not always the right answer. For stable networks where topology rarely changes, the overhead of running the ML model at each election event adds latency without a corresponding benefit. In those cases, using the model once at setup to pick an algorithm and then sticking with it gives most of the benefit at a fraction of the cost. The practical recommendation is therefore not simply "always use dynamic selection," but rather to characterise the rate of topological change in the target environment and choose accordingly.

There are honest limitations to acknowledge as well. The simulation, while more realistic than prior work in that it models bandwidth, packet loss, and node failures, is still a simulation. Real networks have correlated failures, bursty traffic patterns, and routing asymmetries that are hard to capture in a discrete-event model. The dataset was generated synthetically, which means the models have not been validated on traces from real distributed deployments. It is possible that features which matter in practice, such as geographic distribution or cross-datacenter latency, are absent from the feature set entirely. These gaps do not invalidate the results, but they do mean the framework should be treated as a proof of concept rather than a production-ready system.

The deep learning model is also harder to inspect than logistic regression. In a system that makes autonomous decisions affecting cluster stability, opacity is a real concern. An operator who wants to understand why the model chose Raft for a particular configuration cannot get a clean answer from an MLP the way they could from a logistic regression coefficient table. This trade-off between accuracy and interpretability is a recurring tension in applied machine learning, and in critical infrastructure settings it sometimes tips in favour of the simpler, more transparent model even at some cost to performance.

The project was carried out by two people working within the constraints of personal hardware and open-source tools, which put a ceiling on dataset size and the range of configurations we could explore. A larger simulation budget would allow for denser coverage of the parameter space, particularly for edge cases like very high failure rates or extremely sparse topologies where the dataset is currently thinner. These are also the conditions where getting the algorithm selection right matters most, so expanding coverage there would be the most valuable use of additional compute.

A fair amount of self-directed learning was needed to get this work done, since distributed systems and machine learning are not always taught in a way that connects them. Understanding how to build a meaningful evaluation metric, how to handle class imbalance in the training data, and how algorithm-level behaviour translates into learnable features all required going beyond coursework. Graph neural networks came up repeatedly in the literature as a more principled way to handle topology-aware learning, and while we set that aside as future work, it became

clear during this project that keeping up with adjacent fields is not optional if you want to know what the right tools are for a given problem.

# References


[1] H. Garcia-Molina, "Elections in a distributed computing system," *IEEE Transactions on Computers*, vol. C-31, no. 1, pp. 48–59, 1982.

[2] E. Chang and R. Roberts, "An improved algorithm for decentralized extrema-finding in circular configurations of processes," *Communications of the ACM*, vol. 22, no. 5, pp. 281–283, 1979.

[3] D. S. Hirschberg and J. B. Sinclair, "Decentralized extrema-finding in circular configurations of processors," *Communications of the ACM*, vol. 23, no. 11, pp. 627–628, 1980.

[4] G. Tel, *Introduction to Distributed Algorithms*. Cambridge University Press, 2nd ed., 2000.

[5] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC)*, pp. 305–319, USENIX Association, 2014.

[6] L. Lamport, "The part-time parliament," *ACM Transactions on Computer Systems*, vol. 16, no. 2, pp. 133–169, 1998.

[7] N. Dhariwal, "Using machine learning regression model to predict the optimum election algorithm for parallel and distributed computing systems," *International Conference on Smart Technologies, Communication and Robotics (STCR)*, 2023.

[8] A. Favier, J. Besson, M. Arnal, and A. Fraboulet, "A topology aware eventual leader election algorithm for mobile ad hoc networks," in *Proceedings of the 22nd International Conference on Distributed Computing and Networking (ICDCN)*, ACM, 2020.

[9] E. Sakic and W. Kellerer, "Seer: Performance-aware leader election in single-leader consensus," in *Proceedings of the IEEE International Conference on Communications (ICC)*, IEEE, 2021.

[10] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv*, 2016.

[11] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," *arXiv*, 2017.

[12] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 1, pp. 4–24, 2021.

Generated using Undegraduate Thesis LaTeX Template, Version 2.2. Department of Computer Science and Engineering, Bangladesh University of Engineering and Technology, Dhaka, Bangladesh.

This thesis was generated on Wednesday 14th October, 2026 at 8:36pm.